SonarQube — 3–4 YOE Scenario-Based Q&A

1. SonarQube Quality Gate is failing. What will you check?
Ans: First, I check the SonarQube analysis results and identify which Quality Gate condition is failing. I check whether the failure is related to bugs, vulnerabilities, code smells, duplicated code, or code coverage.
Then I review the affected code and verify the Quality Gate thresholds configured for the project. If the issue is valid, I fix the code and run the analysis again.
Simple ga gurthupettuko: Quality Gate fail ayithe → first ye condition fail ayyindo chudu → bugs, vulnerabilities, code smells and coverage check cheyyi → failed code identify cheyyi → Quality Gate threshold verify cheyyi → issue fix chesi analysis malli run cheyyi.
Eng: If the Quality Gate fails → first identify which condition failed → check bugs, vulnerabilities, code smells and coverage → identify the affected code → verify the Quality Gate threshold → fix the issue and run the analysis again.
Example:
mvn sonar:sonar → SonarQube analysis run cheyyi.
SonarQube dashboard → Quality Gate → ye condition failed ayyindo chudu.
Coverage failed ayithe → coverage report generate avutunda check cheyyi.
Issue fix chesi → mvn sonar:sonar → Quality Gate pass ayyinda verify cheyyi.

---

2. SonarQube coverage suddenly dropped. What will you check?
Ans: First, I check the current code coverage and compare it with the previous successful build. Then I verify whether new code was added without sufficient unit tests.
I also check whether the test cases are running successfully and whether the coverage report is being generated and picked up correctly by SonarQube.
Simple ga gurthupettuko: Coverage sudden ga drop ayithe → first current and previous coverage compare cheyyi → new code ki tests unnaya chudu → tests successful ga run avutunnaya check cheyyi → coverage report generate ayyinda verify cheyyi → SonarQube report ni correct ga read chesthunda check cheyyi.
Eng: If coverage suddenly drops → first compare current and previous coverage → check whether new code has tests → verify tests are running successfully → check whether the coverage report is generated → verify SonarQube is reading the report correctly.
Example:
mvn test → unit tests successful ga run avutunnaya chudu.
mvn test jacoco:report → coverage report generate ayyinda check cheyyi.
target/site/jacoco/index.html → coverage percentage chudu.
SonarQube dashboard → previous build coverage tho compare cheyyi.

---

3. SonarQube server is unavailable. What will you check?
Ans: First, I check whether the SonarQube service is running and whether the server is reachable from the browser or from the Jenkins server. Then I check the SonarQube logs for application or database-related errors.
I also verify server resources such as CPU, memory, and disk space. If SonarQube depends on an external database, I check the database connectivity as well.
Simple ga gurthupettuko: SonarQube unavailable ayithe → first service running unda chudu → server reach avutunda check cheyyi → logs check cheyyi → CPU, memory and disk verify cheyyi → database connectivity chudu → issue fix chesi SonarQube access verify cheyyi.
Eng: If SonarQube is unavailable → first check whether the service is running → verify server connectivity → check logs → verify CPU, memory, and disk → check database connectivity → fix the issue and verify SonarQube access.
Example:
systemctl status sonarqube → SonarQube service running unda chudu.
curl http://localhost:9000 → server locally respond avutunda check cheyyi.
df -h → disk space available unda chudu.
free -m → memory usage check cheyyi.
Logs check chesi → actual error identify cheyyi.

---

4. SonarQube is showing false positives. What will you do?
Ans: First, I review the reported issue and confirm whether it is actually a problem in the application code. If the issue is not applicable to the project, I check whether the rule is too strict or incorrectly identifying the code.
Then I review the SonarQube rule configuration and Quality Profile. If appropriate, I mark the issue as false positive or adjust the rule according to the project standards.
Simple ga gurthupettuko: False positive vaste → first issue nijanga problem aa chudu → rule enduku trigger ayyindo identify cheyyi → Quality Profile and rule check cheyyi → project ki applicable kaakapothe false positive ga handle cheyyi → required ayithe rule configuration correct cheyyi.
Eng: If SonarQube reports a false positive → first confirm whether the issue is actually valid → identify why the rule was triggered → check the Quality Profile and rule → handle it as a false positive if it is not applicable → adjust the rule when required.
Example:
SonarQube dashboard → issue open cheyyi → rule details chudu.
Code ni manual ga verify cheyyi → actual problem unda leda check cheyyi.
Quality Profiles → active rule check cheyyi.
Issue valid kaakapothe → issue ni false positive ga mark cheyyi.

5. SonarQube is reporting security vulnerabilities. What will you check?
Ans: First, I review the vulnerability details and identify which code or dependency is causing the security issue. Then I check the severity and understand the recommended remediation.
If the issue is related to a dependency, I verify the dependency version and check whether a secure version is available. For application code issues, I fix the insecure implementation and run the analysis again.
Simple ga gurthupettuko: Security vulnerability vaste → first vulnerability details chudu → affected code or dependency identify cheyyi → severity check cheyyi → dependency ayithe secure version chudu → code issue ayithe secure implementation use cheyyi → analysis malli run cheyyi.
Eng: If SonarQube reports a security vulnerability → first check the vulnerability details → identify the affected code or dependency → check the severity → verify a secure dependency version → fix insecure code when required → run the analysis again.
Example:
SonarQube dashboard → Security → Vulnerabilities → affected issue open cheyyi.
Dependency issue ayithe → mvn dependency:tree → dependency version chudu.
Secure version available ayithe → pom.xml lo version update cheyyi.
mvn test → tests run cheyyi.
mvn sonar:sonar → vulnerability resolve ayyinda verify cheyyi.

---

6. Jenkins pipeline is blocked because of SonarQube Quality Gate. What will you do?
Ans: First, I check the Jenkins console output and identify why the SonarQube Quality Gate failed. Then I open the SonarQube project and check which condition caused the failure.
I fix the underlying code or coverage issue instead of bypassing the Quality Gate. After the correction, I run the pipeline again and verify that the Quality Gate passes.
Simple ga gurthupettuko: Jenkins Quality Gate valla block ayithe → first Jenkins console output chudu → SonarQube lo failed condition identify cheyyi → code or coverage issue fix cheyyi → Quality Gate bypass cheyyakunda pipeline malli run cheyyi → pass ayyinda verify cheyyi.
Eng: If Jenkins is blocked by the Quality Gate → first check the Jenkins console output → identify the failed condition in SonarQube → fix the code or coverage issue → rerun the pipeline without bypassing the Quality Gate → verify that it passes.
Example:
Jenkins → failed build → Console Output → SonarQube Quality Gate result chudu.
SonarQube dashboard → failed condition identify cheyyi.
Issue fix chesi → Jenkins → Build Now → pipeline malli run cheyyi.
SonarQube → Quality Gate → Passed ani verify cheyyi.

---

7. SonarQube Quality Profile is causing unexpected issues. What will you check?
Ans: First, I check which Quality Profile is assigned to the project and compare it with the expected project standard. Then I review recently added, removed, or changed rules in that profile.
I also check whether the project is using the correct language-specific profile. If a rule is causing unnecessary issues, I review it with the team before changing the profile.
Simple ga gurthupettuko: Quality Profile issue vaste → first project ki ye profile assign ayyindo chudu → correct profile aa verify cheyyi → recent rule changes check cheyyi → language-specific profile correct aa chudu → unnecessary rule issue ayithe review chesi required change cheyyi.
Eng: If the Quality Profile causes unexpected issues → first check the profile assigned to the project → verify the correct profile → review recent rule changes → confirm the language-specific profile → review unnecessary rules and make the required change.
Example:
SonarQube → Project → Quality Profiles → assigned profile chudu.
Quality Profile → Active Rules → recently changed rules check cheyyi.
Issue open chesi → which rule triggered it identify cheyyi.
Required ayithe → rule configuration review chesi change cheyyi.

---

8. SonarQube analysis is failing in Jenkins. What will you check?
Ans: First, I check the Jenkins console output and identify whether the failure is related to the SonarQube scanner, authentication, server connectivity, project configuration, or source code.
Then I verify the SonarQube server URL, token or credentials, scanner configuration, and required build tools. I also check whether the Jenkins agent can reach the SonarQube server.
Simple ga gurthupettuko: Jenkins lo SonarQube analysis fail ayithe → first console output chudu → scanner or authentication issue identify cheyyi → SonarQube URL and credentials verify cheyyi → scanner configuration check cheyyi → Jenkins agent nunchi server reach avutunda chudu → issue fix chesi pipeline malli run cheyyi.
Eng: If SonarQube analysis fails in Jenkins → first check the console output → identify scanner or authentication issues → verify the SonarQube URL and credentials → check scanner configuration → verify Jenkins agent connectivity → fix the issue and rerun the pipeline.
Example:
Jenkins → Console Output → SonarQube error identify cheyyi.
curl http://sonarqube:9000 → Jenkins agent nunchi SonarQube reach avutunda chudu.
Jenkins Credentials → SonarQube token correct aa verify cheyyi.
mvn sonar:sonar → analysis manually test cheyyi.

---

9. SonarQube is not showing the latest code analysis. What will you check?
Ans: First, I check whether the latest Jenkins build actually completed the SonarQube analysis successfully. Then I verify the project key, branch, and source code being analyzed.
I also check the Jenkins console output for scanner errors and verify the analysis timestamp in SonarQube. If the wrong branch or project is being analyzed, I correct the configuration.

Simple ga gurthupettuko: Latest analysis kanipinchakapothe → first Jenkins analysis successful aa chudu → project key and branch verify cheyyi → correct source analyze avutunda check cheyyi → console errors chudu → SonarQube timestamp verify cheyyi → wrong configuration unte correct cheyyi.
Eng: If the latest analysis is not visible → first verify that the Jenkins analysis completed successfully → check the project key and branch → confirm the correct source is analyzed → review console errors → verify the SonarQube analysis timestamp → correct the configuration if required.
Example:
Jenkins → latest build → Console Output → SonarQube analysis success aa chudu.
SonarQube → Project → branch correct aa verify cheyyi.
Analysis timestamp → latest build time tho compare cheyyi.
Project key wrong ayithe → Jenkins SonarQube configuration correct cheyyi.

---

10. SonarQube scanner is unable to connect to the SonarQube server. What will you check?
Ans: First, I verify whether the SonarQube server is running and reachable from the machine where the scanner is running. Then I check the configured SonarQube URL and network connectivity.
I also verify firewall rules, proxy settings, DNS resolution, and authentication if required. After fixing the connectivity issue, I run the scanner again.
Simple ga gurthupettuko: Scanner server ki connect avvakapothe → first SonarQube running unda chudu → scanner machine nunchi server reach avutunda check cheyyi → URL verify cheyyi → DNS, firewall and proxy check cheyyi → authentication verify cheyyi → scanner malli run cheyyi.
Eng: If the scanner cannot connect to SonarQube → first verify that SonarQube is running → check connectivity from the scanner machine → verify the server URL → check DNS, firewall, and proxy settings → verify authentication → run the scanner again.
Example:
curl http://sonarqube:9000 → scanner machine nunchi server reach avutunda chudu.
nslookup sonarqube → DNS resolution check cheyyi.
ping sonarqube → basic connectivity check cheyyi.
Scanner URL correct chesi → mvn sonar:sonar → analysis malli run cheyyi.

---

11. SonarQube project is showing no issues even though code has problems. What will you check?
Ans: First, I verify whether the correct source files are being analyzed. Then I check the project key, source directories, exclusions, and branch configuration.
I also verify whether the relevant Quality Profile and rules are active. If files are excluded from analysis or the wrong branch is being analyzed, I correct the configuration and run the analysis again.
Simple ga gurthupettuko: Issues kanipinchakapothe → first correct files analyze avutunnaya chudu → project key and source path verify cheyyi → exclusions check cheyyi → correct branch aa chudu → Quality Profile and rules active unnaya verify cheyyi → configuration fix chesi analysis malli run cheyyi.
Eng: If SonarQube shows no issues even though the code has problems → first verify the correct files are analyzed → check project key and source paths → review exclusions → verify the branch → check active Quality Profile and rules → fix the configuration and run the analysis again.
Example:
SonarQube → Project → Code → expected files kanipistunnaya chudu.
sonar.sources → correct source directory configure ayyinda check cheyyi.
sonar.exclusions → required files exclude ayyaya chudu.
Correct branch select chesi → analysis malli run cheyyi.

---

12. SonarQube analysis is taking too much time. What will you check?
Ans: First, I check the scanner logs to identify which part of the analysis is taking more time. Then I review the project size, number of files, dependencies, and analysis configuration.
I also check whether unnecessary files are included in the analysis and whether the SonarQube server has sufficient CPU, memory, and database performance. I optimize the configuration where required.
Simple ga gurthupettuko: Analysis slow ga unte → first scanner logs chudu → ye part slow ga undho identify cheyyi → project size and files check cheyyi → unnecessary files analyze avutunnaya chudu → SonarQube server resources verify cheyyi → configuration optimize cheyyi.
Eng: If SonarQube analysis is slow → first check scanner logs → identify the slow part → review project size and files → check unnecessary files → verify server resources → optimize the configuration.
Example:
Jenkins → Console Output → scanner logs lo slow step chudu.
du -sh . → project size check cheyyi.
SonarQube configuration → exclusions verify cheyyi.
top → server CPU and memory usage chudu.
Unnecessary files exclude chesi → analysis malli run cheyyi.

---

13. SonarQube token or authentication is failing. What will you check?
Ans: First, I check whether the token is valid and whether it belongs to the correct SonarQube user. Then I verify that the token is configured correctly in Jenkins or the scanner configuration.
I also check whether the user has the required permissions on the project. If the token is expired, revoked, or incorrect, I generate or configure a valid token securely.
Simple ga gurthupettuko: Token authentication fail ayithe → first token valid aa chudu → correct user token aa verify cheyyi → Jenkins credentials check cheyyi → project permissions verify cheyyi → token revoked or invalid aa check cheyyi → valid token configure chesi analysis malli run cheyyi.

Eng: If token authentication fails → first verify that the token is valid → confirm the correct user → check Jenkins credentials → verify project permissions → check whether the token is revoked or invalid → configure a valid token and rerun the analysis.
Example:
Jenkins → Manage Credentials → SonarQube token correct aa chudu.
SonarQube → My Account → Security → token status verify cheyyi.
Project → Administration → Permissions → user access check cheyyi.
Valid token configure chesi → Jenkins build malli run cheyyi.

---

14. SonarQube is showing too many code smells. What will you do?
Ans: First, I review the code smells and identify whether they are caused by a common coding pattern or by specific parts of the application. Then I check the rule responsible for generating them.
I prioritize important issues and fix them based on severity and project standards. I also avoid changing or disabling rules just to reduce the issue count unless there is a valid reason.
Simple ga gurthupettuko: Code smells ekkuva unte → first issues and rules chudu → common pattern identify cheyyi → important issues prioritize cheyyi → code improve cheyyi → valid reason lekunda rules disable cheyyakudadhu → analysis malli run cheyyi.
Eng: If there are too many code smells → first review the issues and rules → identify common patterns → prioritize important issues → improve the code → do not disable rules without a valid reason → run the analysis again.
Example:
SonarQube → Code Smells → issue list chudu.
Issue open chesi → rule details verify cheyyi.
Repeated pattern unte → source code lo aa pattern identify cheyyi.
Code fix chesi → mvn sonar:sonar → code smell count verify cheyyi.

---

15. SonarQube is showing duplicated code. What will you check?
Ans: First, I check which files or code blocks are contributing to the duplication percentage. Then I review whether the duplication is caused by repeated logic, copied code, or similar implementation across modules.
I refactor common logic into reusable methods or components where appropriate. After refactoring, I run the analysis again and verify that the duplication percentage has improved.
Simple ga gurthupettuko: Duplicated code ekkuva unte → first ye files duplicate avutunnayo chudu → repeated logic identify cheyyi → common code refactor cheyyi → reusable method or component create cheyyi → analysis malli run chesi duplication percentage verify cheyyi.
Eng: If duplicated code is high → first identify the files contributing to duplication → find repeated logic → refactor common code → create reusable methods or components → run the analysis again and verify the duplication percentage.
Example:
SonarQube → Measures → Duplicated Lines → affected files chudu.
Source code → repeated logic identify cheyyi.
Common logic ni → reusable method/component ga refactor cheyyi.
mvn sonar:sonar → duplication percentage malli check cheyyi.

---

16. SonarQube branch analysis is not working as expected. What will you check?
Ans: First, I verify whether the correct branch is being analyzed by checking the Jenkins build and SonarQube project. Then I verify the branch configuration and scanner parameters.
I also check whether the branch exists in Git and whether the Jenkins pipeline is checking out the expected branch. After correcting the configuration, I run the pipeline again and verify the branch analysis.
Simple ga gurthupettuko: Branch analysis wrong ga unte → first Jenkins lo correct branch checkout avutunda chudu → SonarQube lo branch verify cheyyi → scanner configuration check cheyyi → Git branch exist avutunda verify cheyyi → configuration fix chesi pipeline malli run cheyyi.
Eng: If branch analysis is not working correctly → first verify the correct branch is checked out in Jenkins → check the branch in SonarQube → verify scanner configuration → confirm the Git branch exists → fix the configuration and rerun the pipeline.
Example:
git branch --show-current → Jenkins agent lo current branch chudu.
git branch -a → required branch exist avutunda verify cheyyi.
Jenkins → Console Output → checkout branch check cheyyi.
SonarQube → Project → correct branch analysis ayyinda verify cheyyi.

---

17. SonarQube scanner is failing because of Java or version compatibility. What will you check?
Ans: First, I check the Java version installed on the Jenkins agent or scanner machine. Then I compare it with the Java version required by the SonarQube scanner and server version.
I also verify the scanner version and build tool compatibility. If there is a version mismatch, I use the supported Java or scanner version and run the analysis again.
Simple ga gurthupettuko: Java version issue vaste → first installed Java version chudu → scanner required version tho compare cheyyi → SonarQube version check cheyyi → scanner and build tool compatibility verify cheyyi → supported version use chesi analysis malli run cheyyi.
Eng: If there is a Java compatibility issue → first check the installed Java version → compare it with the scanner requirement → check the SonarQube version → verify scanner and build tool compatibility → use a supported version and rerun the analysis.
Example:
java -version → installed Java version chudu.
mvn -version → Maven ye Java version use chesthundo verify cheyyi.
Jenkins tool configuration → Java version check cheyyi.
Supported Java version configure chesi → mvn sonar:sonar → analysis malli run cheyyi.

18. SonarQube report is missing test coverage. What will you check?
Ans: First, I check whether the unit tests are actually running and passing. Then I verify whether the coverage tool is generating the required report.
I also check the SonarQube configuration to make sure the correct coverage report path is configured. If the report is missing or the path is incorrect, I fix the configuration and run the analysis again.
Simple ga gurthupettuko: Coverage report missing ayithe → first tests run avutunnaya chudu → tests pass ayyaya verify cheyyi → coverage report generate ayyinda check cheyyi → report path correct aa chudu → SonarQube configuration verify cheyyi → fix chesi analysis malli run cheyyi.
Eng: If the test coverage report is missing → first verify that tests are running → confirm the tests pass → check whether the coverage report is generated → verify the report path → check SonarQube configuration → fix it and rerun the analysis.
Example:
mvn test → unit tests run ayyaya chudu.
mvn jacoco:report → coverage report generate ayyinda check cheyyi.
ls target/site/jacoco/ → report files unnaya verify cheyyi.
SonarQube configuration → coverage report path correct aa chudu.
mvn sonar:sonar → coverage update ayyinda verify cheyyi.

---

19. SonarQube database or storage issue occurs. What will you check?
Ans: First, I check the SonarQube logs to identify whether the issue is related to the database connection, disk space, or database performance. Then I verify whether the database service is running and reachable.
I also check available disk space and database resource usage. I avoid directly modifying or deleting SonarQube database data and follow the proper recovery procedure.
Simple ga gurthupettuko: Database or storage issue vaste → first SonarQube logs chudu → database connection issue aa identify cheyyi → database service running unda check cheyyi → disk space verify cheyyi → database resources chudu → direct data delete cheyyakunda proper recovery follow cheyyi.
Eng: If there is a database or storage issue → first check SonarQube logs → identify database connection issues → verify the database service → check disk space → review database resources → follow the proper recovery process without directly deleting data.
Example:
df -h → SonarQube server disk space chudu.
systemctl status postgresql → database service running unda verify cheyyi.
systemctl status sonarqube → SonarQube service status check cheyyi.
SonarQube logs → database connection error identify cheyyi.
Issue fix chesi → SonarQube service restart chesi access verify cheyyi.

---

20. How will you troubleshoot a SonarQube issue in a production CI/CD pipeline?
Ans: First, I check the Jenkins pipeline output and identify where the SonarQube stage is failing. Then I verify the SonarQube server availability, scanner configuration, authentication, project configuration, and Quality Gate result.
I also check whether the issue is related to code quality, coverage, connectivity, or infrastructure. After identifying the root cause, I fix the issue, rerun the pipeline, and verify that the Quality Gate passes successfully.
Simple ga gurthupettuko: Production pipeline lo SonarQube issue vaste → first Jenkins stage and logs chudu → failure point identify cheyyi → SonarQube server and scanner check cheyyi → authentication and project configuration verify cheyyi → Quality Gate result chudu → code, coverage or infrastructure issue identify cheyyi → fix chesi pipeline malli run cheyyi → Quality Gate pass ayyinda verify cheyyi.
Eng: If there is a SonarQube issue in a production CI/CD pipeline → first check the Jenkins stage and logs → identify the failure point → verify the SonarQube server and scanner → check authentication and project configuration → review the Quality Gate result → identify code, coverage, or infrastructure issues → fix the issue and rerun the pipeline → verify that the Quality Gate passes.
Example:
Jenkins → Failed Build → Console Output → SonarQube stage error chudu.
curl http://sonarqube:9000 → Jenkins agent nunchi SonarQube reach avutunda check cheyyi.
Jenkins Credentials → SonarQube token verify cheyyi.
SonarQube → Project → Quality Gate result chudu.
Issue fix chesi → Jenkins Build Now → pipeline malli run cheyyi.
SonarQube → Quality Gate → Passed ani final ga verify cheyyi.